15-618 Spring 26

Parallel Dynamic Discrete Collision Detection Engine

Names: Bill Wu, Shu Chen Lin

Andrew ID: shouyuaw, shuchen3

Github repo: https://github.com/ShuChenLin/Parallel-DCD-Engine

Web page:https://shuchenlin.github.io/Parallel-DCD-Engine/

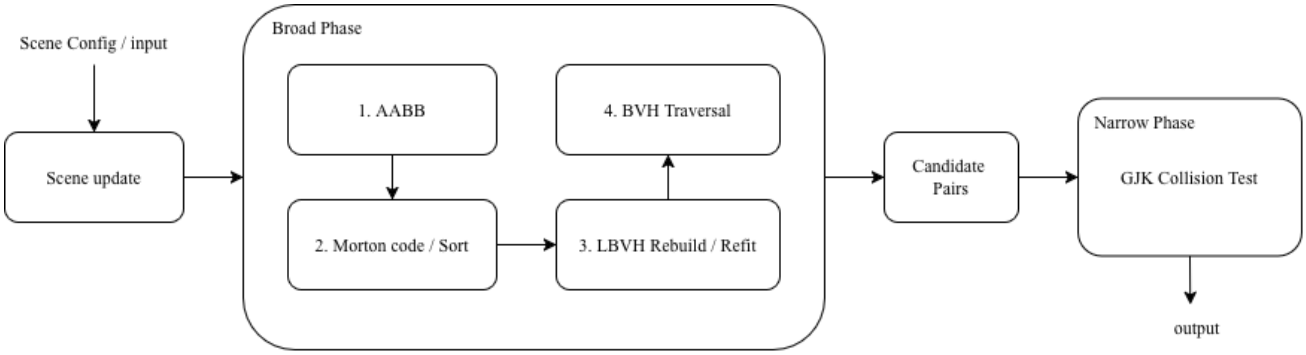
1 SUMMARY

This project focuses on parallel discrete collision detection for large 3D scenes, with an OpenMP implementation, a CUDA implementation, and an OpenGL visualization demo as our main deliverables. The engine uses a BVH broad phase with Morton-code LBVH construction/refit and GJK narrow-phase collision testing across three scenarios, `random_walk`, `two_cluster`, and `avalanche`, which capture different collision-density and workload patterns. We evaluated the OpenMP version on GHC and PSC Bridges-2, scaling up to 128 CPU threads, where our best OpenMP results reached about 41x speedup on `random_walk`, 61x on `two_cluster`, and 41x on `avalanche`. We also have access to an NVIDIA GeForce RTX 2080 GPU on GHC for the CUDA version, achieving about 58x, 48x, and 64x speedup respectively for large data.

2 BACKGROUND

2.1 Problem & Pipeline Overview

Our engine takes as input a set of N convex rigid bodies, each defined by a position and a convex vertex list, and outputs the set of colliding pairs each frame. We solve discrete collision detection per frame instead of continuous collision detection. It addresses the scalability problem with a two-phase pipeline: a broad phase that builds a Linear Bounding Volume Hierarchy (LBVH) [1] from Morton-code-sorted bounding boxes to prune non-colliding pairs in O(N log N) time, followed by a narrow phase that runs the Gilbert-Johnson-Keerthi (GJK) algorithm [2] on the remaining candidates to confirm exact intersections. We evaluate the engine across three scenarios that capture different collision-density patterns: **random_walk**, where bodies move randomly throughout the scene with low collision density; **two_cluster**, where two dense groups of bodies collide head-on, generating a large number of pairs; and **avalanche**, where bodies accumulate into increasingly dense configurations over time.



2.2 Key Data Structures

Structure	Description
AABB	axis-aligned bounding box for individual body
BVHNode	BVH tree node; left/right/parent/object_idx
ConvexShape	vertex list; support function for GJK queries
Morton code	30-bit Z-order code; for sorting

2.3 Key Operations

[Morton Code & Sorting]: Each AABB centroid is normalized to $[0, 1]^3$ and encoded as a 30-bit Z-order (Morton) code. Bodies are then sorted by this code using radix sort, ensuring that spatially nearby bodies are adjacent in memory. A critical property for cache-friendly LBVH construction and traversal.

[LBVH Build (Karras 2012)]: Each internal node independently determines its covered object range and split point using only the sorted Morton code array, with no inter-node dependencies. On subsequent frames, the tree structure is reused and only the bounding boxes are refitted bottom-up via atomic flags[1].

[BVH Traversal (Broad Phase)]: Instead of querying the BVH for each object, we perform an iterative dual traversal over pairs of BVH nodes. The traversal starts from the root pair. If two nodes' bounding boxes are not overlapped, we pruned the entire subtree pair immediately. Keep traverse until we find the leaves being strong candidates. This algorithm avoids redundant per-object root traversals but it’s more challenging to be parallelized.

[GJK Narrow Phase]: Each candidate pair runs the Gilbert-Johnson-Keerthi algorithm to confirm true intersection via iterative simplex refinement, converging in roughly 10–30 iterations. All pairs are processed independently[2].

2.4 Parallelism Analysis

All five stages expose substantial data parallelism. AABB update, Morton code computation, and GJK are embarrassingly parallel. Each body or pair is fully independent. BVH traversal is similarly independent per leaf. LBVH construction is parallel at the node level; the only synchronization required is the bottom-up refit pass. The pipeline stages themselves are sequentially dependent (each stage consumes the output of the previous).

Profiling the sequential baseline at N=500,000 reveals that GJK and traversal dominate execution time:

Stage	random_walk	two_cluster	avalanche
Traverse	168 ms (32%)	829 ms (22%)	175 ms (29%)
GJK	328 ms (61%)	2934 ms (76%)	399 ms (65%)

The main sources of synchronization and scalability loss are BVH refit, candidate pair generation, and irregular work distribution in traversal or narrow phase.

3. APPROACH

3.1 Technologies & Target Machine

OpenMP

- Target: GHC (up to 8 cores), PSC Bridges-2 (up to 128 cores)
- Profiling: perf, per-stage timers reporting per-frame average time for each pipeline stage

CUDA

- Target: NVIDIA GeForce RTX 2080 on GHC
- Profiling: Nsight Compute (ncu), per-stage timers reporting per-frame average time for each pipeline stage

Both implementations share the same sequential C++17 pipeline as a baseline. An OpenGL visualizer built with GLFW provides real-time rendering for demonstration.

3.2 OpenMP Mapping & Algorithm Changes

Parallelizing the DCD pipeline with OpenMP can be done stage by stage. Stages including Morton code computation and GJK narrow phase are easy to parallelize and map cleanly to OpenMP. However, other stages like BVH refit and broad-phase traversal are limited by irregular work and synchronization. Our implementation combines straightforward loop parallelism and some algorithmic changes including a structure-of-arrays hot path, a periodically rebuild policy, and a task-based dual traversal for the broad phase.

3.2.1 Decomposition Strategy

We parallelize each stage independently since there are strong stage-to-stage dependencies. Since the stage costs are also highly uneven, a producer-consumer design would likely leave many cores idle on the cheaper stages. Each stage runs to completion before the next begins, and we focus on maximizing parallel efficiency within each stage.

For optimizations outside of OpenMP, we also switch the data structure to a structure-of-arrays layout. Positions, velocities, and shape types are stored in different arrays and reused across frames. This reduces unnecessary traffic through a larger Body structure during computation and even improves memory locality.

3.2.2 AABB Update, Morton Codes, and GJK

AABB update, Morton code computation, and GJK narrow phase are parallelized straightforwardly with OpenMP loop parallelism. AABB update and Morton code computation are mapped over bodies with `#pragma omp parallel for schedule(static)` since the workload for every node is uniform and these stages are small. GJK is parallelized with dynamic scheduling `#pragma omp for schedule(dynamic, 64)` since the number of iterations varies across pairs. Some confirmed collisions are stored into thread-local vectors and gathered after the parallel session.

3.2.3 Parallel Radix Sort

A four-pass, 8-bit least-significant-digit radix sort over the 30-bit Morton code is implemented. In each pass, every thread builds a thread-level 256-bin histogram. After that, we add up all per-thread histograms, compute the bucket offsets with a prefix sum and scatter the indices into an output buffer. We parallelize the histogram and scatter steps with `schedule(static)` and keep the 256-element prefix sum serial.

In practice, sorting is a relatively small stage in the pipeline. It only cost a few milliseconds even at one million object data. As a result, its speedup is limited not only by the small serial work in each pass, but also by OpenMP overhead and the synchronization cost. It scales reasonably at small thread counts, but the benefit drops once the per-thread workload becomes too small.

3.2.4 Parallel LBVH Build and Refit

On rebuild frames, we construct the LBVH [1] with the Karras-style formulation. In the current implementation, leaf initialization and internal-node topology construction are both parallelized with `schedule(static)`. Each internal node writes its own child links and parent pointers, so no synchronization is required during topology construction itself.

After the topology is built, bounding boxes still need to be propagated bottom-up through the tree. The same refit operation is also used on refit-only frames, where the BVH topology is reused and only node bounds are updated. We implement refit with a standard atomic-flag propagation scheme[7]. First, all leaf AABBs are updated in parallel. Then each leaf thread walks upward through the parent chain. At an

internal node, the first arriving child increments the flag and exits. The second arriving child knows both children are ready, merges their AABBs, updates the node's leaf_count, and continues upward.

This design avoids explicit locks and keeps the implementation simple. However, profiling shows that refit does not scale well at high thread counts. The problem is not only the atomic increment. As threads walk upward, they eventually converge near the root. At that point, very little parallel work remains.

3.2.5 Dual-Traversal Broad Phase

As discussed in 2.3, we chose dual-traversal over an independent root-to-leaf BVH query for every object. An entire subtree might be pruned whenever the two nodes' AABBs do not overlap. If both nodes are leaves, it emits a candidate pair. Otherwise, it keeps grinding and comparing until a leaf. This is a huge improvement on performance however a challenging part to be parallelized.

Dual traversal produces highly irregular and unpredictable work. Some node-pair tasks are pruned immediately while others expand into larger subproblems. To parallelize this stage, we start from the root pair and repeatedly split it into smaller subtree-pair tasks until task count is larger or equal to core nums. The task list is then processed with `#pragma omp parallel for schedule(dynamic, 1)`, and each thread writes its candidate pairs into a thread-local vector to avoid contention on a shared output buffer. After the parallel region, the thread-local vectors are concatenated into the final candidate-pair list.

For dense scenes, especially `two_cluster`, simply having enough tasks (equal to core nums) is not always sufficient. Load imbalance is a big deal, therefore, we added an optional weighted-splitting mode for `two_cluster`. In this mode, if a subtree pair still looks too large based on its `leaf_count`, we keep splitting it into smaller tasks before parallel execution begins. This produces a more balanced task list and significantly improves traversal load balance in `two_cluster`.

3.3 CUDA Mapping & Algorithm Changes

Each stage of the pipeline maps to a dedicated CUDA kernel, with work distributed at the finest possible granularity:

3.3.1 Decomposition Strategy

stage	Thread assignment
AABB update / Morton code	1 thread = 1 body
Radix Sort	Thrust library
LBVH build	1 thread = 1 internal node
Refit	1 thread = 1 leaf (propagates upward)
BVH traversal	1 thread = 1 leaf (body)
GJK narrow phase	1 thread = 1 candidate pair

All kernels use a block size of 256 threads, except GJK which uses 32 (explained in 3.3.5). Radix sort is handled by Thrust.

3.3.2 AABB Update, Morton Code, and Sort

AABB update and Morton code computation are embarrassingly parallel. Each body is independent. Both are launched with `gridDim = ⌈N/256⌉` blocks of 256 threads; thread i processes body i. Radix sort (Morton codes → sorted indices) is delegated to `thrust::sort_by_key`, which provides a highly optimized GPU sort without requiring custom kernel development.

3.3.3 LBVH Build and Refit

LBVH construction follows the Karras 2012 algorithm[1]. Each of the N−1 internal nodes is assigned one thread; threads determine their node's children by computing the range of Morton codes they cover using `delta()` (common prefix length) comparisons[7]. All internal nodes are built independently in a single kernel launch. Refit uses an atomic-flag approach: each leaf thread walks upward toward the root, and the first sibling to arrive at a parent sets an atomic flag and exits; the second sibling merges AABBs and continues. This guarantees every parent is processed exactly once without a second kernel pass [7].

3.3.4 BVH Traversal

Each thread is assigned one leaf (query body) and traverses the BVH with a private register-based stack (`int stack[64]`). Register stacks avoid shared memory overhead and allow full occupancy (85 ~ 93% achieved). To reduce `atomicAdd` contention when writing collision pairs to the global output buffer, we use warp ballot reduction: threads within a warp use `__ballot_sync + __popc` to count how many threads in the warp have a hit, then one elected thread issues a single `atomicAdd` for the entire warp.

3.3.5 GJK Narrow Phase

Each thread is assigned one candidate pair and runs the full GJK iterative loop independently [2]. The key algorithmic change from the sequential version is a shared memory vertex prefetch: at kernel launch, each block cooperatively loads all convex hull vertices for the bodies it will process into shared memory. This reduces GJK global memory reads from 1,536 per pair to 24 (one coalesced load per body at startup).
The trade-off is shared memory footprint: each block requires ~9 KB (32 threads × 24 vertices × 12 bytes per float3). This constrains block size to 32 threads. Larger blocks would consume more shared memory per block and reduce the number of blocks that fit per SM. At block size 32, the GPU fits 7 blocks per SM, achieving 21.9% occupancy (limited by shared memory capacity).

3.3.6 Data Transfer and Output

All GPU buffers (positions, vertices, BVH nodes, pair lists) are allocated once at scene initialization and reused across frames. Per-frame data transfer consists only of updated positions and velocities (N × 2 × 12 bytes), avoiding redundant uploads of static geometry.

Collision pair output uses a zero-copy pinned buffer: h_narrow_pinned is allocated with cudaMallocHost and mapped into device address space. The narrow-phase kernel writes results directly into this buffer, eliminating a separate device-to-host cudaMemcpy for the pair list each frame.

3.4 Optimization Iterations

3.4.1 OpenMP Iterations

[Structure of Arrays]: As we mentioned above, this reduced memory traffic in AABB update and GJK setup, and improved locality. On PSC at 128 threads, this change improved the overall speedup by roughly 15% across the three scenarios. The absolute gain was modest, but it was consistent and remained in the final implementation.

[Periodic Rebuild]: We reused the BVH topology across most frames and only performed a full rebuild periodically. This avoided recomputing Morton codes, sorting, and topology construction on every frame. In our measurements, this improved absolute runtime by about 20% compared with rebuilding every frame. However, profiling also showed that the refit operation itself still scales poorly at high thread counts, so this policy reduced average cost per frame without removing the main bottleneck.

[Weighted Dual Traversal]: To improve load balance in dense scenes such as `two_cluster`, we added a weighted task-splitting strategy using subtree leaf_count as a rough estimate of task size. This produced a more balanced task frontier and improved the two_cluster speedup at 128 threads from about 55x to 64x. Applying the same strategy to all scenarios added too much overhead, so the final version enables it only for two_cluster.

[Level-Order Refit]: We also tested level-order refit and other non-atomic refit variants in order to reduce synchronization cost. In practice, these versions reduced rather than improved scalability: the speedup at high thread counts dropped compared with the atomic bottom-up refit. The extra barriers, coordination, and memory traffic outweighed the benefit of removing atomics. As a result, the atomic refit remained in the final implementation even though it is still one of the main scaling bottlenecks.

3.4.2 CUDA Iterations

[GJK Vertex Access]

The initial GJK kernel reads vertex data from global memory on every iteration of its 64-step loop. To improve memory coalescing, we first switched from Array-of-Structs (AoS) to Struct-of-Arrays (SoA) vertex layout. This caused a 9 ~ 13× regression: GJK pairs are randomly assigned, so adjacent threads access different bodies. SoA coalescing only helps when adjacent threads access adjacent bodies, which is not the case here. We instead prefetched all vertices into shared memory at kernel launch. Each block cooperatively loads its pairs' vertices once, and the GJK loop reads exclusively from shared memory, reducing global reads from 1,536 to 24 per pair. This yielded 1.6 ~ 2.8× GJK speedup across scenarios.

[GJK Occupancy Tuning]

Nsight Compute showed GJK occupancy at only 12.5%: with block size 128, each block consumes 36 KB of shared memory, and the SM's 65 KB capacity fits only one block. We progressively reduced block size: 128 → 64 → 32, which increased blocks per SM from 1 → 3 → 7, raising occupancy from 12.5% to 21.9% and yielding cumulative 5 ~ 8% GJK improvement per step. We also tested replacing shared memory with __ldg() read-only cache, which raised occupancy to 87 ~ 95% but caused a 2 ~ 3× regression due to L1 thrashing under dense collision scenarios. Shared memory's guaranteed locality outweighs the latency-hiding benefit of higher occupancy for this access pattern.

[BVH Memory layout]

Nsight Compute showed traverse kernel Compute SM utilization at only 7.8% in two_cluster despite 93% occupancy, indicating most cycles were stalled on memory. We attempted to improve cache locality by reordering BVH nodes into BFS breadth-first layout, reasoning that upper-tree nodes would benefit from cache-line sharing across threads.

This caused a 26 ~ 40% regression. BVH traversal is depth-first, so in BFS layout each step jumps by 2^depth in memory, destroying path locality. The original Karras ordering implicitly places nearby bodies' ancestor nodes at nearby indices, giving natural spatial locality that BFS eliminates. We reverted and accepted random node access as a fundamental property of tree traversal.

3.5 Prior Work and References

We did not start from an existing codebase. All code was implemented from scratch. The LBVH construction algorithm follows Karras [1] and the accompanying "Thinking Parallel" blog [7], with the Morton code bit-interleaving routine adapted from [7]. For the GPU memory layout and BVH pipeline structure, we referenced ToruNiina's CUDA LBVH implementation [3] and surveyed existing physics engines including bullet3 [10] and ReactPhysics3D [9] to understand common design patterns. The GJK narrow-phase implementation is based on Lindemann [2], Yazdani & Wachs [3], and the dyn4j [5] and Winter [6] tutorials.

4. RESULTS

4.1 Experimental Setup

We evaluate both implementations across three problem sizes (N = 50,000, 500,000, and 1,000,000) and three scenarios: random_walk, two_cluster, and avalanche. The primary metric is average detection time per frame, broken down by pipeline stage. Speedup is reported relative to the single-threaded sequential baseline.

The three scenarios represent a spectrum of collision densities:

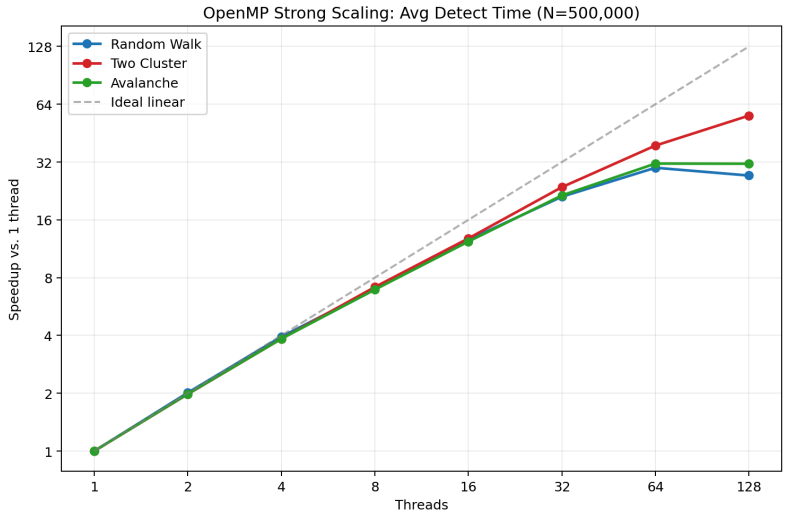
- **random_walk**: Bodies move with random velocities uniformly throughout the scene, producing sparse, uncorrelated collisions. This represents the low-density baseline.
- **two_cluster**: Two dense groups of bodies move toward each other. When the clusters collide, the number of candidate pairs explodes, stressing the narrow phase. This is the worst-case workload.
- **avalanche**: Bodies fall and accumulate into a dense pile, representing a moderate-density workload typical of real physics simulations.

4.2 Overall Speedup (Avg time per frame)

4.2.1 OpenMP

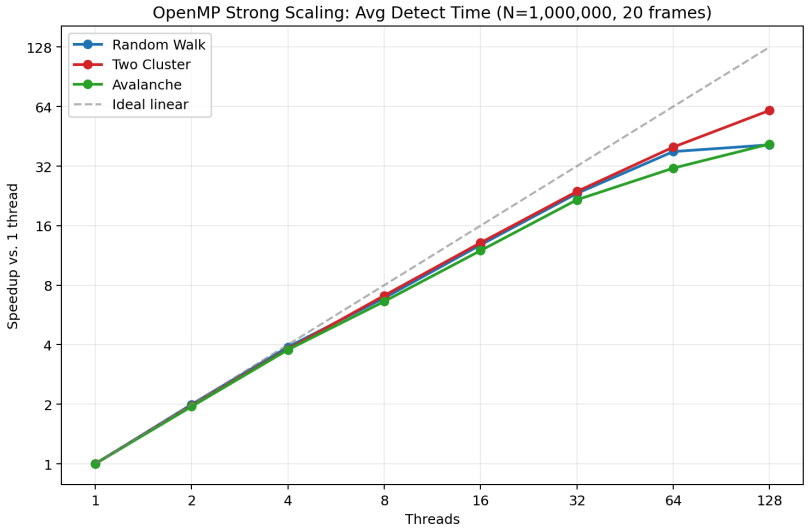
N=500K, time(ms)

Scenario	1T	2T	4T	8T	16T	32T	64T	128T
random_walk	782.49	388.74	198.79	112.20	62.20	37.05	26.18	28.75
two_cluster	8301.71	4195.56	2161.34	1161.73	648.38	350.57	212.89	148.66
avalanche	1079.05	544.28	280.86	155.72	87.51	50.30	34.35	34.42



N=1M, time(ms)

Scenario	1T	2T	4T	8T	16T	32T	64T	128T
random_walk	2706.09	1355.62	694.98	392.70	211.58	116.28	71.39	66
two_cluster	31896.9	16193.1	8399.83	4498.43	2432.89	1339.38	798.8	521.15
avalanche	3754.90	1928.70	996	567.2	313.73	173.65	120.14	90.72



4.2.2 CUDA

N=500K, Frame=50

Scenario	Sequential	CUDA	Speedup
random_walk	534.2 ms	9.2 ms	58.3x
two_cluster	3837.9 ms	79.6 ms	48.2x
avalanche	610.7 ms	9.5 ms	64.3x

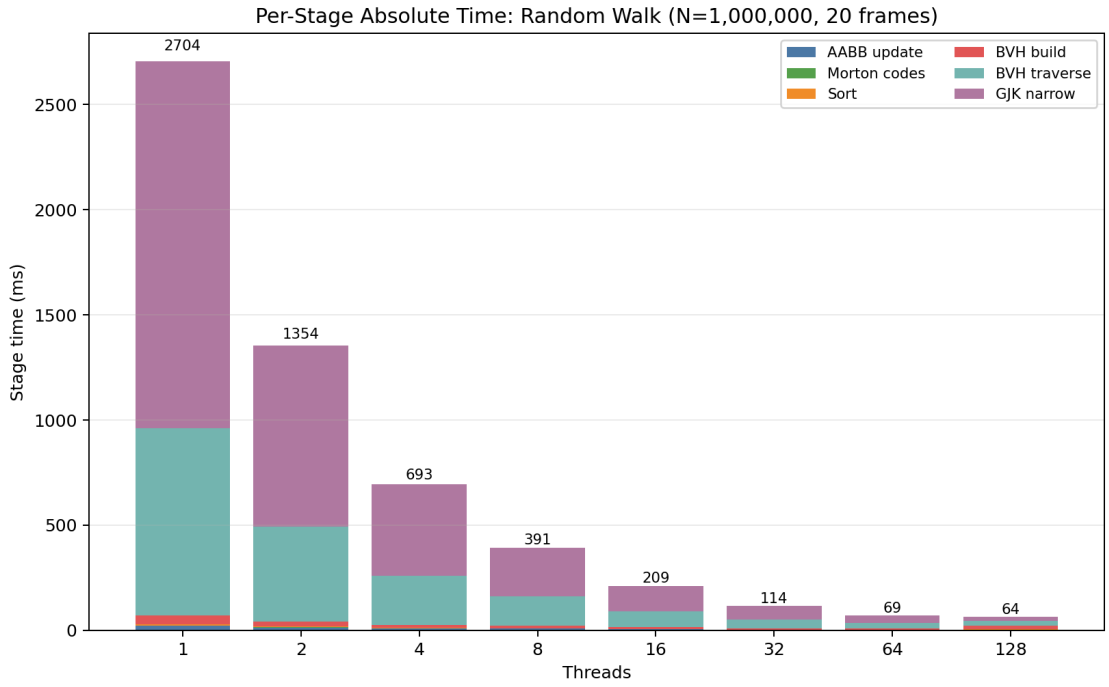
N=1M, Frame=50

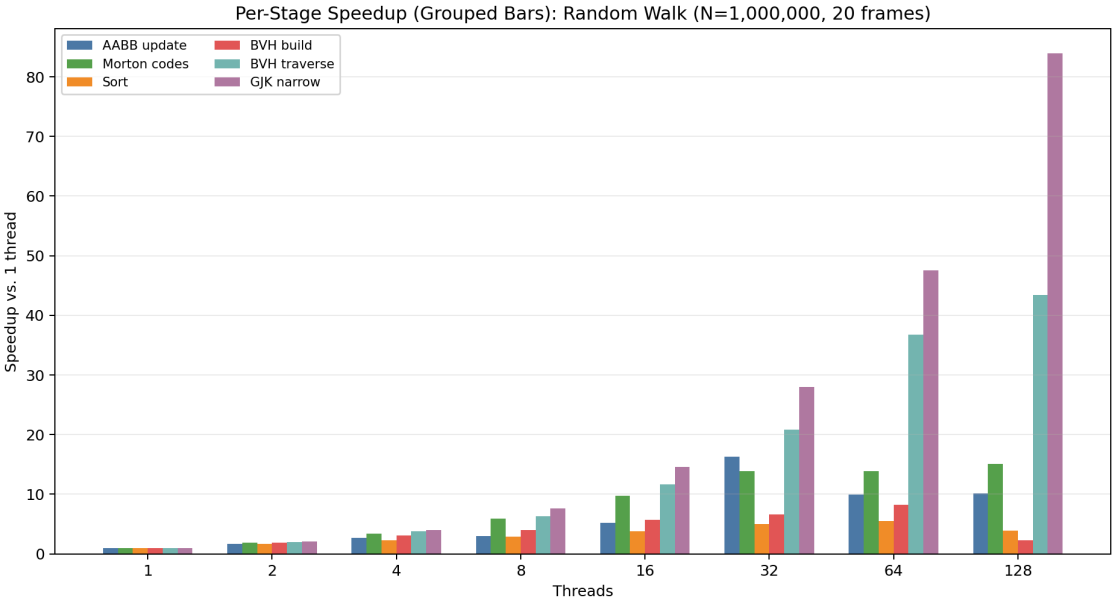
Scenario	Sequential	CUDA	Speedup
random_walk	1866.8 ms	33.6 ms	55.5x
two_cluster	15363.3 ms	359.0 ms	42.8x
avalanche	2185.1 ms	48.4 ms	45.1x

4.3 Per-Stage Breakdown

4.3.1 OpenMP

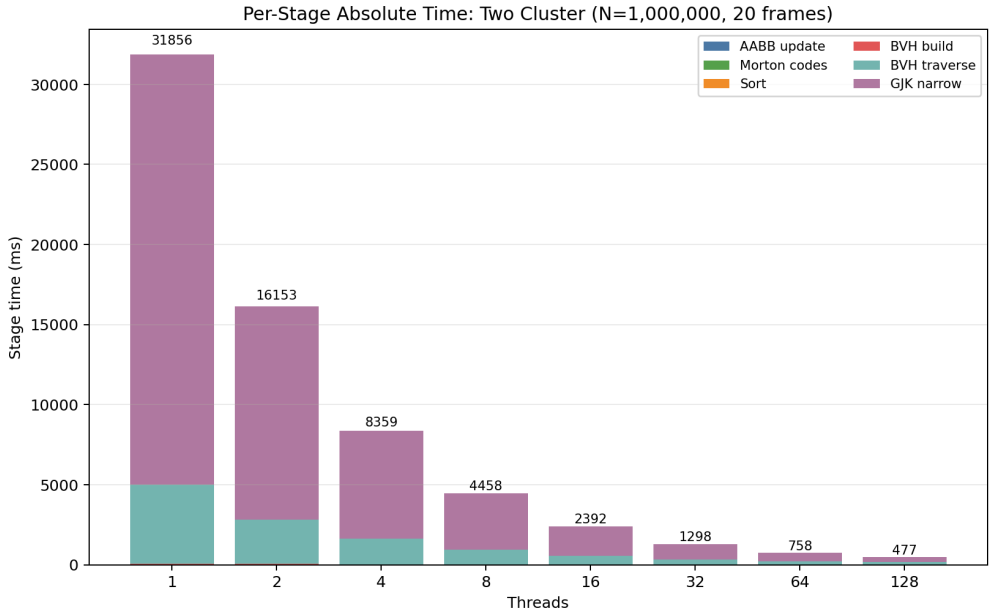
All per-stage measurements in this section are collected on PSC using the `N=1,000,000` workload and thread counts from 1 to 128. For each scenario, we present one figure showing absolute stage time and one figure showing per-stage speedup.

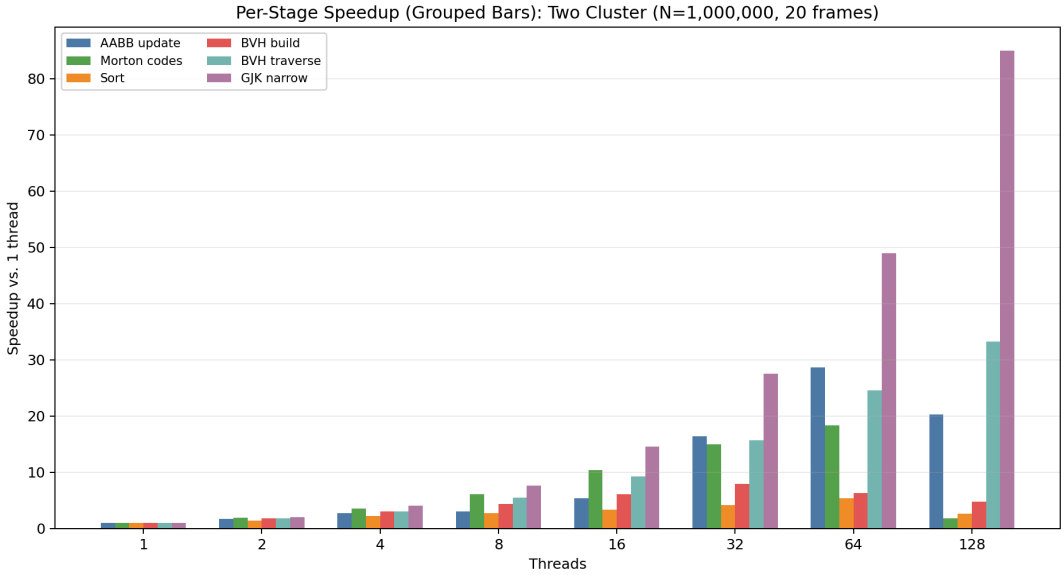




[random_walk]: The code scales well overall, with average detect time decreasing from about 2706 ms at 1 thread to 66 ms at 128 threads, corresponding to roughly 41x speedup. The largest contributors at low thread counts are GJK narrow and BVH traverse, and both stages scale strongly as more threads are

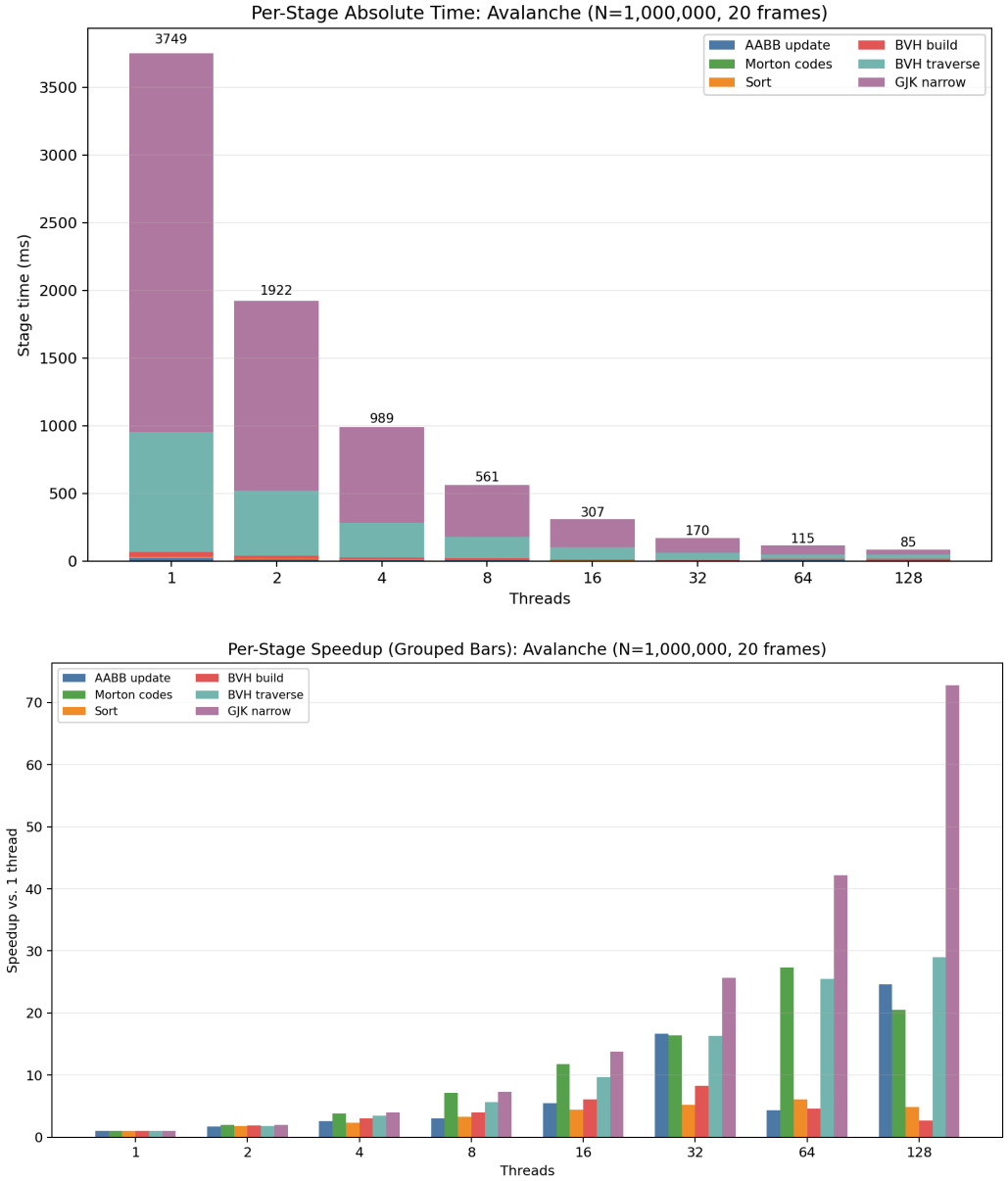
added. In particular, GJK reaches about 84x speedup and traversal reaches about 43x speedup at 128 threads, showing that the collision-heavy parts of the pipeline parallelize effectively. However, the overall speedup does not continue increasing at the same rate beyond 64 threads. That's probably because BVH build becomes the main bottleneck: its time drops from 42.49 ms at 1 thread to only 19.06 ms at 128 threads, much worse than the other major stages. This indicates that the remaining scalability limit is not GJK or traversal load balance, but the poor high-thread scaling of BVH build/refit.





[two_cluster]: This has the best speedup among all three scenarios, with average detect time decreasing from about 31897 ms at 1 thread to 521 ms at 128 threads, or roughly 61x speedup. This behavior is expected because `two_cluster` generates an extremely dense collision region, so both BVH traverse and GJK narrow dominate the runtime and provide abundant parallel work. At 1 thread, traversal takes about 4919 ms and GJK takes about 26868 ms, and both stages continue to shrink substantially as the thread

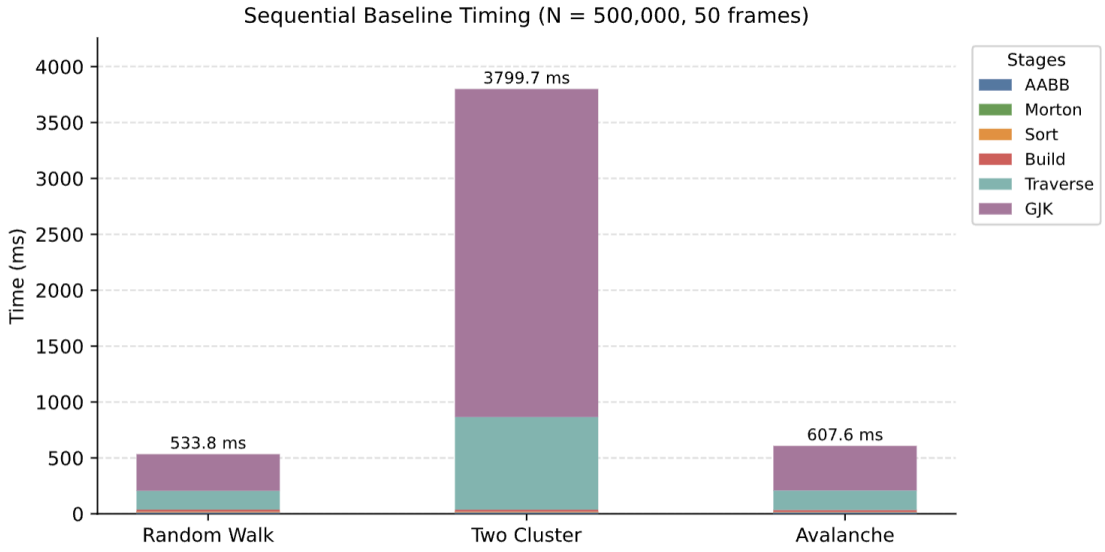
count increases. GJK reaches about 85x speedup and traverses about 33x speedup at 128 threads, showing that the heaviest parts of the workload parallelize well. Although BVH build still scales worse than traversal and GJK, it remains a relatively small fraction of the total time in this scenario. As a result, `two_cluster` benefits the most from high thread counts, and its scaling confirms that the weighted traversal strategy is effective for balancing dense broad-phase work.

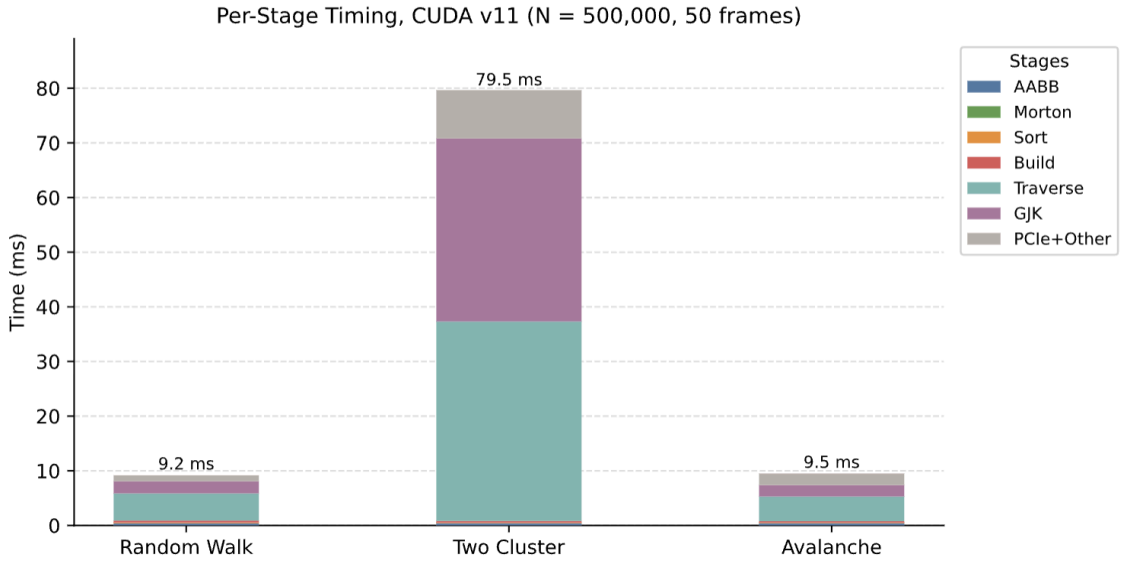


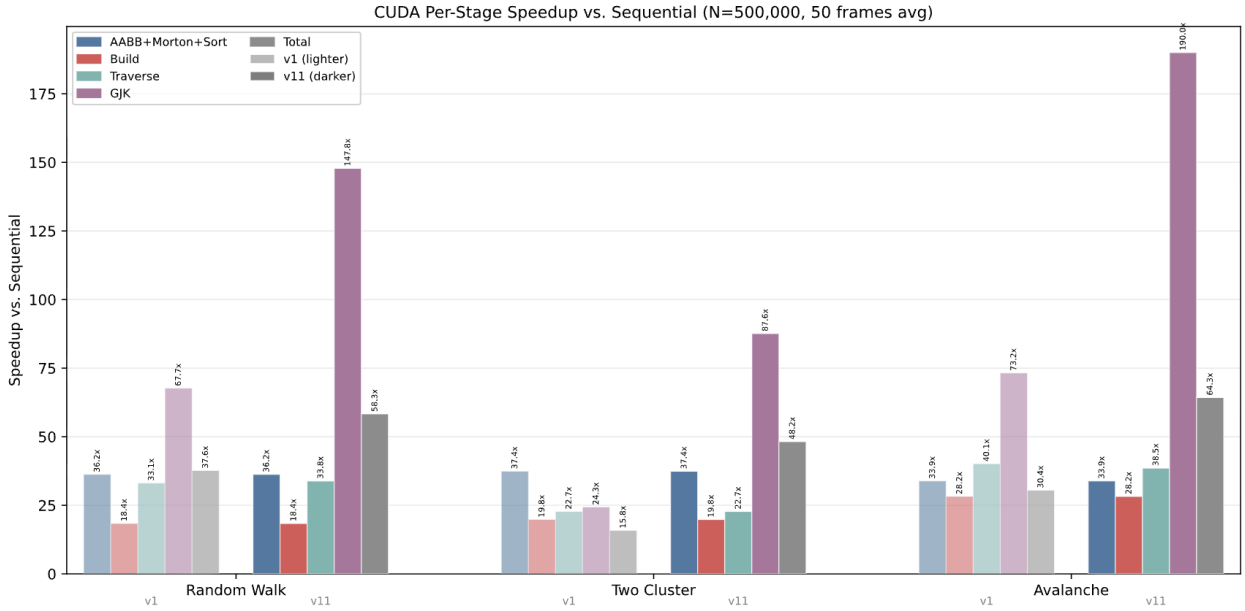
[Avalanche]: Our implementation also scales well in this scenario, with average detect time decreasing from about 3755 ms at 1 thread to 91 ms at 128 threads, corresponding to roughly 41x speedup. At low thread counts, the runtime is dominated by GJK narrow and BVH traverse, which shrink when more threads are added. GJK reaches about 73x speedup at 128 threads, while traversal reaches about 29x, showing that both stages parallelize decently. However, unlike `two_cluster`, the overall speedup begins to flatten earlier because BVH build becomes a major cost at high thread counts. Its time decreases from 38.22 ms at 1 thread to only 14.19 ms at 128 threads, much worse than the scaling of GJK and traversal.

This means that in avalanche, the limiting factor at high core counts is no longer the collision-testing work itself, but the poor scalability of BVH build/refit, which takes up a much larger fraction of the remaining runtime as the other stages shrink.

4.3.2 CUDA







Three observations follow from this breakdown:

GJK dominates the sequential baseline. Across all three scenarios, GJK accounts for 61 ~ 76% of sequential total time. Since each pair is fully independent, GJK parallelizes well: CUDA final version achieves 88 ~ 190x per-stage speedup through massive thread-level parallelism combined with shared memory vertex prefetch, which reduces global memory reads from 1,536 to 24 per pair.

Traverse becomes the new bottleneck in later versions. After GJK is accelerated, BVH traversal rises to 45 ~ 54% of total time, with a comparatively modest speedup of 23 ~ 39x. Each thread follows a different path through the tree, resulting in non-coalesced memory access and frequent L1 cache evictions. This is analyzed further in Section 4.5.

AABB update and LBVH build are no longer bottlenecks. Together they account for less than 15% of total time across all scenarios, with a consistent 18–30x speedup. These stages are embarrassingly parallel and memory-access-friendly, so they scale well without further tuning.

two_cluster is a special case: the sheer volume of collision pairs: up to 17.5M per frame, which makes both Traverse and GJK dominant. In the final version, each accounts for roughly 42 ~ 46% of total detection time, a distribution not seen in the other two scenarios.

4.4 Bottleneck Analysis: OpenMP

To better understand the bottleneck of our program, we used perf as a profiling tool to measure different metrics and summarize the result. All tests are run on a GHC machine with 8 cores.

4.4.1 Load Balance

```
DCD_LOAD_BALANCE=1 ./dcd_omp -n 500000 -f 3 -t 8 --no-validate
```

This records per-thread traversal work, emitted candidate pairs, GJK calls, and per-thread execution time, allowing us to compare work imbalance against actual time imbalance.

Scenario	BVH Work Imbalance	BVH Time Imbalance	GJK Call Imbalance	GJK Time Imbalance
random_walk	~2.0x	~1.09x	~1.02x	~1.00x
two_cluster	~2.5x	~1.00x	~1.04x	~1.00x
avalanche	~2.2x	~1.12x	~1.04x	~1.00x

Per-thread instrumentation is used for load-balance testing. The results show that GJK is already very well balanced in all three scenarios: both the number of calls and the per-thread time stay very close across threads. BVH traversal shows a much larger imbalance in work count, since some threads visit more node pairs and emit more candidate pairs than others. However, the traversal time imbalance is much smaller, which suggests that dynamic scheduling hides most of this irregularity. Overall, these results show that the remaining speedup limit is not caused by severe load imbalance, but more likely by BVH build/refit and the irregular overhead of the broad phase.

4.4.2 Memory and Cache Behavior

```
perf stat -e \
      cycles,instructions,cache-references,cache-misses,L1-dcache-loads,L1-dcac
      he-load-misses,LLC-loads,LLC-load-misses,LLC-stores,LLC-store-misses,dTLB
      -loads,dTLB-load-misses,branches,branch-misses \
      ./dcd_omp -n 1000000 -f 20 -t 8 --no-validate
```

Threads	IPC	Cache Miss Rate	L1 Miss Rate	LLC Load Miss Rate	Branch Miss Rate	dTLB Miss Rate
8	1.5	11.12%	1.00%	8.69%	6.28%	0.02%

The result suggests that the code is not purely compute-bound. The low L1 and dTLB miss rates indicate that the hot loops still have decent local reuse. However, the LLC load miss rate is higher, which suggests that the larger working set of the broad phase and narrow phase still puts pressure on the last-level cache. The branch miss rate is also nontrivial, which is consistent with the irregular control flow in BVH traversal and GJK. Overall, the perf results support the view that the OpenMP implementation is limited by a mix of irregular memory access and branch-heavy traversal logic rather than by raw arithmetic throughput alone.

4.4.3 Remaining Scalability Bottlenecks

We would say the remaining scalability bottlenecks depend on the workload. In large and dense cases such as `two_cluster`, BVH traversal and GJK dominate the total runtime. GJK scales very well, but traversal does not improve as cleanly because it still involves irregular task sizes, heavy candidate-pair

generation, and memory-intensive broad-phase work. Even though dynamic scheduling hides most of the severe time imbalance, traversal remains one of the main costs in the largest workloads.
In lighter cases such as random_walk and avalanche, traversal and GJK shrink much faster as thread count increases. As a result, BVH build/refit becomes a larger fraction of the remaining runtime and turns into the dominant bottleneck at high thread counts. Its bottom-up atomic propagation is simple and correct, but it scales poorly because all upward walks eventually converge near the root, where little parallel work remains. Sorting also shows weaker scaling at high thread counts, but its absolute runtime is small, so it does not dominate the end-to-end runtime.
This version is closer to the actual behavior you measured:

- `two_cluster`: traversal/GJK are still the big costs
- `random_walk` / `avalanche`: build/refit becomes dominant after the other stages shrink

4.5 Bottleneck Analysis: CUDA

In the final implementation of CUDA at N = 500,000, BVH traversal and GJK narrow phase together account for over 85% of total detection time across all scenarios (reference 4.3.2). AABB update, LBVH build, and sort are no longer bottlenecks, each contributing less than 15% of total time. The remaining performance gap relative to theoretical GPU throughput is explained by two distinct bottlenecks in the two critical kernels.

4.5.1 GJK Smem-limited Occupancy

Nsight Compute reports GJK occupancy at 21.9% (7 warps/SM) and Compute SM utilization at 13 ~ 17%. The binding constraint is shared memory: each block requires 9 KB for vertex prefetch, and the SM's 65 KB capacity fits at most 7 blocks. With only 7 warps in flight per SM, any memory stall leaves the SM largely idle. Table X shows how occupancy and performance vary across configurations:

Configuration	smem/block	Blocks/SM	Occupancy	Compute SM	GJK Time (tc)
Block size=128	36KB	1	12.5%	13.5%	42.6 ms
Block size=64	18KB	3	18.75%	16.8%	35.3 ms
Block size=32	9KB	7	21.9%	18.3%	33.5 ms
__ldg()	0KB	7	88%	6.3%	111.6 ms

Despite progressive block size reduction from 128 → 32, occupancy is capped at 21.9% by shared memory capacity. Removing the prefetch in favor of __ldg() raised occupancy to 87 ~ 95% but caused a 2 ~ 3× regression with 28 warps competing for L1 that vertex cache lines were evicted before reuse, and Compute SM dropped to 6.3%. Shared memory's guaranteed locality outweighs the benefit of higher occupancy, making 21.9% the practical ceiling for GJK on this hardware.

4.5.2 Traverse Memory-bound

Despite high occupancy (88–93%), traverse_kernel shows low Compute SM utilization across all scenarios: 7.8% for two_cluster, 22% for avalanche, and 30.5% for random_walk. Threads stall on memory rather than execute useful work.

Scenario	Occupancy	Compute SM	L1/TEX	DRAM
random_walk	92%	30.5%	81.3%	13%
two_cluster	93%	7.8%	49.4%	38.9%
avalanche	88%	22%	73.8%	18.5%

The root cause is the stack-based DFS traversal: each thread follows a data-dependent path through the BVH and accesses different GBVHNode entries with no spatial regularity, preventing memory coalescing. Collision density amplifies the effect. In random_walk, sparse body distribution causes early traversal termination, and L1/TEX utilization reaches 81.3% with only 13% DRAM pressure. In two_cluster, dense clusters force deep traversals that exceed L1 capacity, pushing DRAM utilization to 38.9% and Compute SM down to 7.8%. We attempted BFS memory reordering to co-locate sibling nodes, but as described in Section 3.4, this destroyed depth-first locality and worsened performance. Without a fundamentally different traversal strategy, irregular per-thread memory access remains the hard limit of the broad phase.

The primary bottleneck of our CUDA pipeline is memory latency, not compute throughput. GJK is constrained by shared memory capacity (21.9% occupancy ceiling), while BVH traversal suffers from irregular, data-dependent access that defeats coalescing. Both are structural properties of the collision detection workload.

4.6 OpenMP vs. CUDA

Although the absolute sequential times differ (the two implementations ran on different machines), both pipelines cost roughly the same on a GHC machine, so we compare speedup over each implementation's own sequential baseline.
At N = 1,000,000, OpenMP (128 threads) achieves 41×, 61×, and 41× on random_walk, two_cluster, and avalanche, while CUDA achieves 56×, 43×, and 45×. CUDA wins on the sparser scenarios, but on the dense two_cluster OpenMP's speedup is higher. High collision density benefits 128-thread CPU parallelism: BVH traversal and GJK scale well across cores, masking poor build/refit scaling. The same density hurts CUDA: irregular DFS traversals push Compute SM down to 7.8%, and ~17.5 M output pairs impose significant transfer overhead. The GPU advantage narrows as collision density rises.

Overall, GPU remains the better target: it wins on the two more common scenarios and achieves higher peak speedup. The two_cluster result shows that a well-threaded CPU can be competitive under heavy collision load, and memory-bound traversal is the primary bottleneck to address.

5. REFERENCES

1. [https://diglib.eg.org/items/71edb707-2be1-4d84-8e25-9162cd9a59e9] Karras, T. (2012). Maximizing Parallelism in the Construction of BVHs, Octrees, and k-d Trees. Proc. High-Performance Graphics (HPG 2012), pp. 33–37. Eurographics Association.
2. [https://www.medien.ifi.lmu.de/lehre/ss10/ps/Ausarbeitung_Beispiel.pdf] Lindemann, P. (2010). The Gilbert-Johnson-Keerthi Distance Algorithm. LMU Munich.
3. [https://www.sciencedirect.com/science/article/pii/S001046552500270X] Yazdani, A., & Wachs, A. (2025). Performance optimization of GJK collision detection in discrete element simulations. Computer Physics Communications, 316, 109768.
4. [https://github.com/NVIDIA/thrust]NVIDIA Corporation, Thrust Library.
5. [https://dyn4j.org/2010/04/gjk-gilbert-johnson-keerthi/] dyn4j blog — "GJK (Gilbert–Johnson–Keerthi)"
6. [https://winter.dev/articles/gjk-algorithm/] Winter's "Collision Detection in 2D/3D" blog series
7. [https://developer.nvidia.com/blog/thinking-parallel-part-iii-tree-construction-gpu/] T. Karras, "Thinking Parallel, Part III: Tree Construction on the GPU," NVIDIA Developer Blog, 2012.
8. [https://github.com/ToruNiina/lbvh] CUDA LBVH Implementation
9. [https://github.com/DanielChappuis/reactphysics3d] ReactPhysics3D: open-source C++ physics engine; surveyed for BVH and GJK pipeline structure.
10. [https://github.com/bulletphysics/bullet3] bullet3: open-source physics engine with GPU collision detection surveyed for broad/narrow-phase design patterns.